

ОТЧЁТ

прохождения проектно-образовательной практики

обучающегося очной формы обучения 1 курса группы ИОП-ИТ-25/2

образовательной программы высшего образования, интегрированной с образовательной программой среднего общего образования, по специальности «Проектирование, разработка и управление сложными информационными системами»

Заковенко Романа Александровича

Место прохождения практики:

АНО ДПО «Образовательные технологии Яндекса»,
РФ, г. Москва, ул. Тимура Фрунзе, д. 11, к. 2,
(дистанционно)

Руководитель практики от Университета:

Семёнов Михаил Евгеньевич,
научный руководитель,
направление «Финансовая математика и финансовые технологии», НЦ ИТиИИ

Сроки прохождения практики: с «29» июня 2026 г. по «24» июля 2026 г.

Оценка: _____

«__» июля 2026 г.

_____/М.Е. Семёнов

ОГЛАВЛЕНИЕ

1. ВВЕДЕНИЕ	2
2. ОСНОВНАЯ ЧАСТЬ	2
2.1 Вводные определения, термины и аббревиатуры	2
2.2 ОПРЕДЕЛЕНИЕ АРХИТЕКТУРЫ ПРОГРАММНОЙ РАЗРАБОТКИ	6
2.3 ОПРЕДЕЛЕНИЕ СРЕДСТВ РАЗРАБОТКИ И ВЫБОР МОДЕЛЕЙ.....	6
2.4 СБОР ДАТАСЕТА.....	7
2.5 ОБУЧЕНИЕ МОДЕЛЕЙ	7
2.6 ЛОКАЛЬНОЕ РАЗВЕРТЫВАНИЕ	8
3. ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ	8

1. ВВЕДЕНИЕ

С развитием искусственного интеллекта и машинного обучения появляются новые отрасли, использующие технологии этой области. Одной из ключевых таких отраслей является Physical AI. Цель этого направления – объединить наработки в области робототехники и передовые алгоритмы AI для создания мультидоменных систем, способных легко адаптироваться к изменениям рабочей среды.

В рамках реализации национальных проектов, направленных на технологическое развитие России, Яндекс осуществляет подготовку кадров в этой области, в том числе путём обучения заинтересованных школьников и студентов. В рамках проектной практики Яндекс поставил задачу разработать комплекс программных средств, которые могут стать «точкой входа» для начинающих специалистов.

2. ОСНОВНАЯ ЧАСТЬ

Перед началом работы была проведена установочная встреча с представителем Яндекса Андреем Кармацким для постановки задач кейса. Для выполнения этих задач была сформулирована цель: разработать три чекпоинта моделей машинного обучения для робота-манипулятора SO-101, а также набор программного обеспечения для развёртывания этих моделей на локальном компьютере.

2.1 Вводные определения, термины и аббревиатуры

Термин	Значение
МО (ML)	Машинное обучение (Machine Learning) – класс алгоритмов и моделей компьютерных наук, которые по входным массивам данных, целевым меткам или сигналам подкрепления из среды выявляют скрытые закономерности и настраивают собственные параметры для решения поставленной задачи без жёсткого программирования правил, причём одна и та же модель может адаптироваться под разные наборы данных и смежные сценарии использования.

Термин	Значение
ИИ (AI)	Искусственный интеллект (Artificial Intelligence) – система, алгоритм или программный комплекс, который по входным разнородным данным, целевым ограничениям и контекстуальным инструкциям имитирует когнитивные функции человека, обучается и принимает автономные решения для решения широкого спектра прикладных задач, причём одна и та же интеллектуальная система может оперировать множеством разнородных запросов и динамически адаптироваться к изменяющимся условиям среды.
VLM	Vision Language Model – класс моделей машинного обучения, которые по входным изображениям или видеопотоку и сопутствующей текстовой инструкции предсказывают текстовое описание, ответы на вопросы или разметку объектов для решения поставленной в запросе задачи.
VLA	Vision Language Action – класс моделей машинного обучения, которые по входным изображениям сцены с камер, текстовой инструкции и показаниям датчиков вращения моторов предсказывают последовательность перемещений робота для решения поставленной в запросе задачи.
ACT	Action Chunking Transformer – класс моделей машинного обучения, представляющий собой модификацию архитектуры трансформера для предсказания чанка – последовательности движений моторов для решения одной заранее поставленной задачи.
SO-101	Настольный робот-манипулятор с пятью степенями свободы.
Фреймворк (Framework)	Программная платформа, определяющая структуру программной системы, буквально «каркас», на котором строятся последующие слои программного обеспечения.
Инференс (Inference)	Inference – класс процессов и технологических этапов в машинном обучении, при которых предобученная модель ИИ принимает на вход новые, ранее не встречавшиеся данные, выполняет прямое прохождение сигнала через свои настроенные веса и предсказывает целевой результат или управляющие команды для решения поставленной в запросе задачи.
LeRobot	Открытый фреймворк, разработанный компанией Hugging Face и предназначенный для сбора данных, обучения моделей физического ИИ и их развёртывания на произвольных роботах.
SmolVLA	VLA-модель на пятьсот миллионов параметров, представляющая собой тандем из небольшой VLM и обучаемого слоя «экспертов» для управления роботом по текстовому запросу.
CUDA	Программно-аппаратная платформа корпорации NVIDIA для вычислений на видеокарте.

Термин	Значение
Телеоперация	Класс методов и технологических процессов в робототехнике, при которых оператор-человек в реальном времени транслирует свои движения через ведущее устройство (leader) на ведомое (follower).
BM (VM)	Виртуальная машина (Virtual Machine) – программная система или программно-архитектурное решение, которое по входным инструкциям гостевой операционной системы и запросам к изолированным виртуальным ресурсам эмулирует работу физического компьютера на базе аппаратного обеспечения хост-машины под управлением гипервизора.
SSH	Secure Shell – сетевой протокол прикладного уровня, который по входным зашифрованным командам пользователя, данным аутентификации и конфигурационным файлам обеспечивает безопасное удалённое управление операционной системой, туннелирование трафика и передачу файлов между компьютерами через незащищённую сеть.
Docker	Платформа контейнеризации, которая упаковывает приложение и его зависимости в изолированные переносимые контейнеры для их надёжного запуска в любой операционной системе.
Docker-образ	Неизменяемый шаблон с инструкциями, который содержит код приложения, среду выполнения, библиотеки и системные инструменты, необходимые для развёртывания и запуска работающего контейнера.
API	Application Programming Interface – набор чётко определённых правил, протоколов и инструментов, который по входным структурированным запросам от одного программного компонента позволяет ему безопасно взаимодействовать, обмениваться данными и вызывать функции другого приложения или сервиса.
JavaScript	Высокоуровневый язык программирования динамической типизации.
WebRTC	Web Real Time Communication – открытый стандарт и набор сетевых протоколов, который по входным запросам JavaScript API позволяет браузерам и мобильным приложениям передавать потоковое аудио, видео и произвольные данные напрямую между устройствами без участия промежуточного сервера.
VO (VE)	Виртуальное окружение (Virtual Environment) – изолированная директория в файловой системе, которая по входным командам установки пакетов позволяет изолировать зависимости, библиотеки и версии интерпретатора конкретного проекта от глобальной операционной системы и других проектов на этом же компьютере, причём внутри одного и того же окружения могут гарантированно воспроизводиться специфичные версии модулей без риска возникновения конфликтов зависимостей в процессе разработки.

Термин	Значение
Python-пакет	Python-пакет (Python package) – способ структурирования и распространения кода, представляющий собой каталог с модулями и специальным файлом инициализации, который по входным инструкциям импорта в исходном коде программы позволяет разработчикам повторно использовать, логически группировать и подключать сторонний или собственный функционал.
uv	Высокопроизводительный менеджер пакетов и проектов для Python.
ResNet-18	Глубокая свёрточная нейросеть из 18 слоёв с архитектурой остаточных связей, которая по входным изображениям предсказывает вероятности их принадлежности к определённым классам, извлекает высокоуровневые визуальные признаки или формирует компактные векторные представления для решения задач компьютерного зрения.
Датасет (Dataset)	Структурированный массив или упорядоченная совокупность аннотированных данных, которая по входным запросам скриптов обучения или алгоритмов валидации обеспечивает модель машинного обучения примерами для настройки её внутренних весов и проверки точности работы.
Хост (Host)	Любое сетевое устройство, подключённое к компьютерной сети, которое по входным сетевым пакетам и запросам других узлов предоставляет вычислительные ресурсы, запускает службы или обеспечивает доступ к данным в рамках протоколов TCP/IP.
Гипервизор (Hypervisor)	Программное обеспечение или аппаратный микрокод, который по входным инструкциям гостевых операционных систем управляет физическими ресурсами хост-машины и изолирует их друг от друга для создания и одновременного запуска нескольких независимых виртуальных машин.
Bash-скрипт	Текстовый файл, содержащий последовательность команд оболочки Bash и логических операторов, который по входным аргументам командной строки или системным событиям автоматически выполняет цепочку системных операций, администрирует процессы операционной системы и обрабатывает файлы.
JSON	Текстовый формат обмена данными, который по входным структурированным парам «ключ-значение» и массивам обеспечивает легко читаемое человеком и машиной представление информации для её хранения, конфигурации систем или передачи между различными приложениями через API.
Abblation-исследование	Метод анализа архитектур машинного обучения, основанный на поочерёдном удалении, отключении или изменении отдельных компонентов системы.

2.2 Определение архитектуры программной разработки

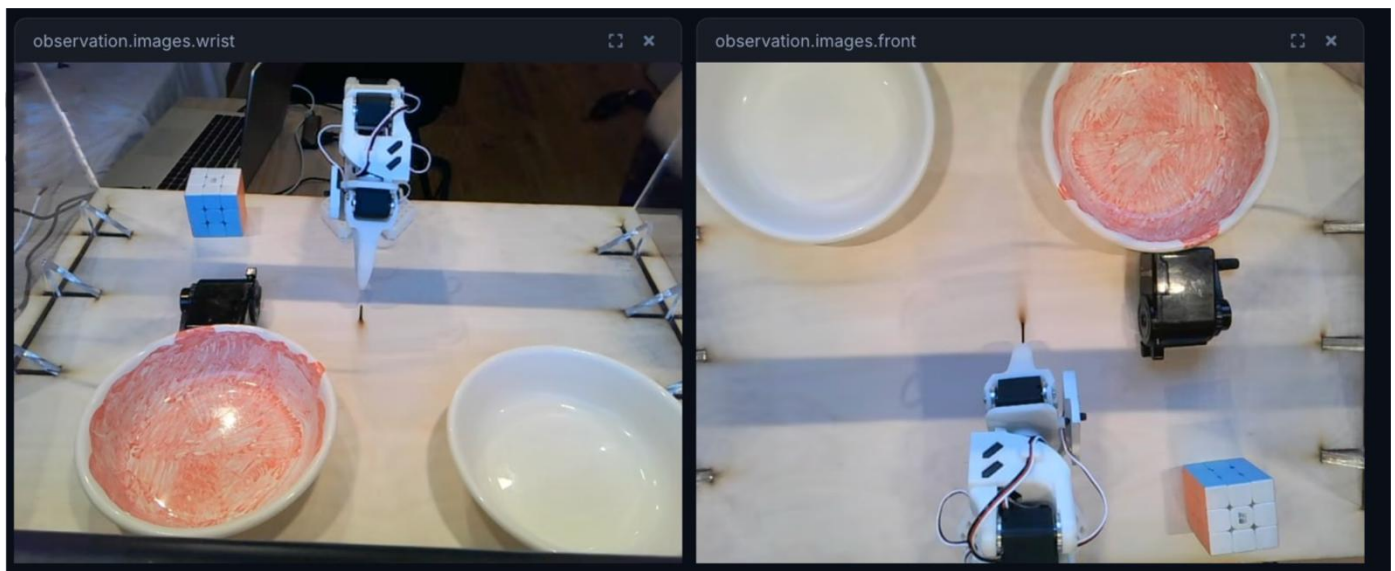
Одной из задач, поставленных перед командой, была разработка архитектуры проекта. В распоряжении команды находились два робота-манипулятора SO-101 в конфигурации leader/follower: первый был оснащён захватом типа «клешня», а второй – рукояткой для оператора. При включении режима телеоперации follower повторял движения leader, которым управлял человек.

Изначально обучение и инференс модели планировалось выполнять на удалённом сервере НТУ «Сириус» с операционной системой Ubuntu 24.04 LTS и видеокартой NVIDIA A5000. Однако в целях обеспечения безопасности на удалённой VM был открыт только один порт для доступа по SSH. Существовала техническая возможность удалённого управления роботом через SSH-туннель, однако задержка оказалась слишком большой для корректной работы системы. Поэтому было решено обучать модели на сервере, а запускать их локально на имеющихся ноутбуках.

2.3 Определение средств разработки и выбор моделей

В первоначальное техническое задание, предоставленное Яндексом, помимо прочего, входил запуск симулятора IsaacSim для сбора синтетических данных, используемых при обучении моделей. Для этого необходимо было запустить Docker-образ с экземпляром IsaacSim и передавать видеопоток на имеющиеся ноутбуки через WebRTC. Однако после изучения документации и проведения тестирования стало ясно, что симулятор плохо поддерживает работу на удалённой виртуальной машине. Поэтому было решено использовать виртуальное окружение uv и Python-пакет lerobot с дополнительными зависимостями.

Модели должны были научиться поднимать три предмета: кубик Рубика, мячик для пинг-понга и красную тарелку, а затем перекладывать их в отдельную белую тарелку независимо от их положения. Для этого были установлены две камеры: одна – непосредственно над сценой, а вторая – перед полигоном, на возвышении.



Для выполнения этой задачи были выбраны модели VLA и ACT. Модель VLA состоит из двух частей: VLM-части, которая отвечает за обработку текстовых инструкций и потока изображений с камер, и action-эксперта, который обрабатывает данные и предсказывает движения сервоприводов. Для нашей работы была выбрана предобученная модель SmolVLA с 500 000 000 параметрами.

Обучение модели происходило следующим образом: 400 000 000 параметров VLM-части замораживались, поскольку обработка текста и изображений уже работала достаточно хорошо и не требовала изменений, а оставшиеся параметры action-эксперта обучались. Об обучающей выборке будет сказано ниже.

В свою очередь, АСТ работает по принципу имитационного обучения. В качестве основы модель использует ResNet-18 для определения объектов интереса и обучается взаимодействовать только с ними.

2.4 Сбор датасета

Из-за невозможности запуска симулятора на удалённом сервере было принято решение собирать датасеты в реальных условиях, используя имеющиеся у нас ноутбуки. Для этого применялись менеджер окружений `uv` и фреймворк `lerobot` с дополнительными пакетами `feetech` и `smolvla`. Всего было собрано десять датасетов:

- шесть датасетов по 50 эпизодов и один из 100 эпизодов для переноса кубика Рубика в тарелку;
- один датасет из 100 эпизодов для переноса мячика для пинг-понга в тарелку;
- один датасет из 50 эпизодов
- и один из 100 эпизодов для перемещения одной тарелки в другую.

Все датасеты и статистика по ним загружены на страницу `sirius-lerobot` на платформе Hugging Face. Во время записи эпизодов в рабочем пространстве размещались различные предметы, а также изменялось освещение. Это позволило сделать выборку более разнообразной и, как следствие, повысить устойчивость модели к помехам.

2.5 Обучение моделей

Обучение моделей проводилось на сервере, предоставленном НТУ «Сириус». Сервер требовал дополнительной настройки, поскольку гипервизор QEMU скрывал некоторые процессорные флаги хост-системы. Из-за этого отдельные Python-пакеты, такие как TensorFlow и PyTorch, не могли корректно работать.

Для дообучения VLA-модели использовалась предобученная модель `smolvla_base` с 500 миллионами параметров. Как было сказано выше, часть модели, отвечающая за обработку изображений и текста, замораживалась, а обучался только эксперт по движениям со 100 миллионами параметров.

Стоит отметить, что базовая модель, поставляемая в составе LeRobot, была обучена с использованием данных с трёх камер. Проблема несоответствия количества камер решалась путём переопределения входных признаков.

Гиперпараметры обучения SmolVLA представлены ниже:

- Продолжительность обучения – 40 000 шагов;
- Размер батча – 16;
- Оптимизатор – AdamW, скорость обучения – 1×10^{-4} ;
- Планировщик скорости обучения – 1000 шагов разогрева с последующим косинусным затуханием до 2.5×10^{-6} ;
- Модель получает одно наблюдение и предсказывает последовательность из 50 действий – поворотов сервоприводов;
- Чекпоинты сохраняются каждые 5000 шагов.

Модели класса АСТ обучаются решению одной конкретной задачи методом имитационного обучения, поэтому они являются достаточно легковесными и содержат около 50 миллионов параметров. Наши модели обучались на тех же датасетах со следующими гиперпараметрами:

- Продолжительность обучения – 100 000 шагов;
- Размер батча – 8;
- Оптимизатор – AdamW, скорость обучения 1×10^{-5} , коэффициент затухания весов – 1×10^{-4} ;
- Скорость обучения – постоянная;
- Визуальный энкодер – предобученная сеть ResNet-18;
- Используется VAE (Variational Autoencoder) – вариационный автоэнкодер, позволяющий закодировать «стиль» выполнения действия. Например, один и тот же предмет можно взять немного выше или ниже, при этом оба варианта будут правильными.

2.6 Локальное развёртывание

Для записи датасетов и локального развёртывания моделей использовался личный ноутбук с Ubuntu 24.04 LTS, процессором AMD Ryzen 7 8845HS и 32 ГБ оперативной памяти. Запись датасетов и запуск моделей выполнялись в том же uv-окружении с помощью собственных Bash-скриптов.

Скрипт развёртывания задаёт системные переменные, содержащие идентификаторы, порты и пути к файлам калибровки ведомого и ведущего манипуляторов. После этого пользователю, от имени которого был запущен скрипт, предоставляются права доступа к каталогу `~/cache`, где по умолчанию хранятся JSON-файлы калибровки, и к файлам устройств манипуляторов в системном каталоге `/dev`. Обычно эти файлы имеют пути `/dev/ttyACM0` и `/dev/ttyACM1`.

Во время записи датасетов периодически возникала проблема: пакеты данных от сервоприводов иногда не поступали в программу, из-за чего она завершалась с ошибкой. Поэтому было решено внести изменения в исходный код LeRobot: если утилите не удавалось считать данные с одного из сервоприводов, она повторяла запрос ещё три раза.

Из-за различий в архитектуре АСТ и SmolVLA для локального инференса последней использовался метод RTC (Real Time Chunking). Он позволял снизить влияние высокой задержки инференса при запуске модели на процессоре без использования видеокарты и обеспечивал более плавное управление манипулятором.

3. ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ

В итоге были отобраны пять чекпоинтов моделей АСТ и SmolVLA: по одному чекпоинту АСТ и SmolVLA для переноса кубика Рубика, по одному чекпоинту АСТ и SmolVLA для переноса мячика для настольного тенниса, а также один чекпоинт SmolVLA для переноса тарелки.

В ходе тестирования было установлено, что после успешного захвата искомого предмета моделям всегда удавалось переместить его в нужное место. При этом модель АСТ захватывала предмет в среднем с четвёртой попытки, а SmolVLA – со второй.

Помимо этого, было проведено ablation-исследование способности SmolVLA выполнять текстовые инструкции, отличающиеся от тех, которые использовались при её обучении. Поскольку VLA-модель благодаря VLM-части может принимать на вход текстовые инструкции, была

предпринята попытка передать модели инструкцию, соответствующую другой задаче. Например, SmolVLA, обученной переносу кубика Рубика, предлагалось взять мячик для настольного тенниса. Около 90 % таких запусков завершились неудачно. Это можно объяснить тем, что в процессе обучения VLA-модель не училась распознавать и захватывать произвольные предметы, а осваивала управление роботом при взаимодействии с конкретными объектами, представленными в обучающей выборке. Предполагается, что увеличение объёма и разнообразия обучающей выборки может повысить точность захвата произвольных объектов.

Кроме того, был создан репозиторий проекта на платформе GitHub, в котором опубликованы четыре Bash-скрипта для инференса моделей и записи датасетов, два JSON-файла с данными калибровки ведущего и ведомого манипуляторов, а также полная инструкция по подготовке окружения, подключению оборудования и запуску программных компонентов. Таким образом, была создана полноценная «точка входа» для начинающих специалистов: размещённые материалы позволяют воспроизвести рабочее окружение и приступить к работе с роботами-манипуляторами SO-101 и фреймворком LeRobot.

Репозиторий проекта на GitHub: <https://github.com/NightyStudios/sirius-lerobot>